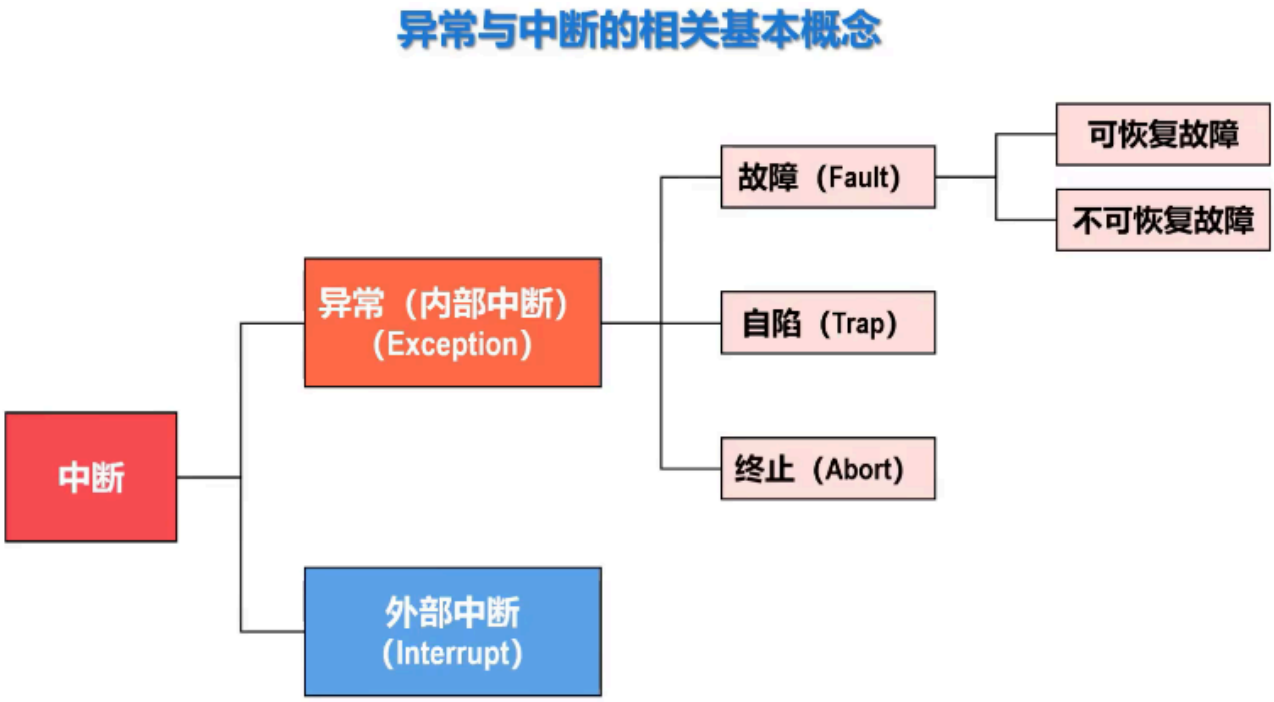
6.21 异常与中断——概述

考纲定位：计算机组成原理 · 第 6 章 中央处理器（6.21） | 重要度 ★★★★ | 常见考法：选择题/大题（异常分类、断点位置、软硬件分工）

一句话核心：异常是同步的内部事件，中断是异步的外部事件；CPU 只负责响应与断点，具体处理由 ISR 完成

跨章定位：通用中断分类、8259A、单级/多重中断见 8.7 中断的概念、8.9 中断请求、8.10 单级中断处理流程、8.11 多重中断的处理流程；本节聚焦 CPU 视角的异常/中断机制

异常与中断是CPU暂停当前程序流、转去处理紧急或意外事件、处理完再返回原断点的控制转移机制，本质是"程序流被劫持"——来自CPU内部叫异常（内部中断），异步来自CPU外部叫中断（外部中断）



一、本质与统一视角：劫持与恢复

无论异常还是中断，CPU都要解决两个核心问题：**在哪断、怎么回。**断点必须"精确"，即保存的PC值能让程序恢复后行为完全一致；处理完必须通过恢复断点来继续执行原程序流 同步与异步是区分二者的根本维度：

- **异常（内部中断）是同步的**，由当前正在执行的指令直接引发，离开这条指令就不会发生，CPU必须停下来处理
- **中断（外部中断）是异步的**，由外部设备发出，与当前指令毫无关系，可能发生在任何时刻 对响应时机的规定是：CPU只在**一条指令执行结束时**才查询中断请求，绝不会在指令执行中途响应，这保证了指令执行的原子性

二、四大类型的宏观辨析

异常（内部中断）内部又分为三类，与中断（外部中断）合称四种控制转移机制，关键差异体现在"断点指向哪条指令"与"是否可恢复"

类型	触发源	断点位置	典型例子	恢复行为	408关键词
故障 Fault	当前指令执行中出错	**当前指令**（可重新执行）	缺页、TLB缺失、除数为0、保护错	修复后**重新执行原指令**	断点指向==当前指令==
自陷 Trap	程序**主动**安排的指令	**下一条指令**	系统调用（trap/syscall）、条件自陷、断点调试	返回下一条指令继续	事先安排的"陷阱"
终止 Abort	硬件严重故障	不确定	掉电、存储器线路错、总线奇偶错	**终止进程**，无法恢复	不可恢复
中断（外部中断） Interrupt	外部设备（异步）	**下一条指令**（因CPU在指令边界响应）	键盘、定时器、I/O完成、DMA结束	返回被打断处继续	可屏蔽/不可屏蔽
记忆锚点：**Fault是"这条指令没做完"，Trap是"这条指令做完了故意拐弯"，Abort是"做不下去了"，Interrupt是"跟这条指令无关"**					
自陷与故障的关键区别：自陷是预先安排的，断点记录的是自陷指令的**下一条**地址；故障的断点记录的是发生故障的**当前指令**地址，以便修复后重新执行					

外部中断按可屏蔽性分为两类：

- **可屏蔽中断（INTR）**：通过中断屏蔽字可以屏蔽掉，正常的外设请求走这条路
- **不可屏蔽中断（NMI）**：优先级最高，任何时候都必须响应，用于掉电等紧急事件

三、响应与处理过程：硬件与软件的职责边界

整个处理过程分为两大阶段，硬件与软件各有明确分工

硬件自动完成（中断隐指令）

CPU在响应中断时**自动执行**的内部操作，没有对应的机器指令，程序员无法干预，包含三件事

1. **关中断**：将中断允许标志清0，防止后续步骤被新中断打断
2. **保存断点**：将PC（下一条指令地址）压栈或存入指定寄存器

3. **引出中断服务程序**：通过中断向量识别中断源，取ISR入口地址送PC，跳转过去 **中断向量表**是实现第3步的核心数据结构：每个中断源有唯一的中断类型号，类型号作为索引查中断向量表，得到对应中断服务程序的入口地址

软件完成（中断服务程序ISR）

由操作系统编写的一段真正的程序，从ISR第一条指令开始

1. **保护现场**：将ISR中会用到的通用寄存器内容压栈
2. **开中断**（可选）：允许更高优先级中断嵌套进来
3. **中断处理**：执行具体的服务逻辑
4. **关中断**（如果开了）
5. **恢复现场**：从栈中弹回寄存器内容
6. **中断返回**：执行IRET等指令，恢复PC和PSW 硬件与软件之所以要这样切分，原因在于：**保存断点**只有PC一个值，硬件做最快；**保护现场**要保存哪些寄存器取决于ISR代码，硬件不知道，只能软件做

四、高频考点与易混点辨析

考点1：中断隐指令 vs 中断服务程序的操作划分

2010年第32题经典陷阱：把"保护现场"误判为硬件自动完成。正确记忆：**硬件只做关中断、保存断点、引出ISR三件事；保护现场、开中断、恢复现场都是软件的活**

考点2：故障类异常能否重新执行原指令

- **缺页、TLB缺失**：可以，OS调入页面后从故障指令重新执行
- **非法操作码、保护错**：不能，必须终止进程
- **浮点除0**：架构可自定义，可用特殊值（NaN/Inf）表示后继续，也可终止

考点3：不同异常的断点位置

- Fault（可修复的）：断点 = **当前指令地址**
- Trap、Interrupt：断点 = **下一条指令地址**
- Abort：断点无意义

考点4：中断响应的三个必要条件

CPU能响应中断必须同时满足

1. CPU处于开中断状态（IF=1）
2. 中断源有未被屏蔽的有效请求
3. 当前指令已执行完毕

考点5：自陷与故障的本质区别

故障是**被动**的、意外的；自陷是**主动**的、预先安排的。系统调用就是最典型的自陷——程序员写trap指令是有意请求内核服务，不是程序出错了